AdaptED: Markov Decision Processes and Deep Reinforcement Learning for Adaptive Educational Content Recommendation

Vai Patankar

Pennsylvania State University | vqp5224@psu.edu

Abstract

This paper presents **AdaptED**, an adaptive educational content recommendation system grounded in Markov chain theory and Markov Decision Processes (MDPs). Current educational recommender systems give the same content to every student regardless of their knowledge level. AdaptED treats recommendation as a sequential probabilistic decision problem. At each step, a learning agent observes a student knowledge state, selects one of 150 possible content recommendations, and receives a reward shaped by Vygotsky's Zone of Proximal Development. A Dueling Deep Q-Network learns to approximate the optimal action-value function through repeated interaction with a student simulator governed by an Ebbinghaus forgetting process. Training runs for 1,000 episodes across four student profiles and shows consistent knowledge gains for fast and average learners, while revealing that struggling students face a fundamental limit by their stochastic learning and forgetting rates.

1. Introduction

Online education platforms host millions of videos, yet their recommendation algorithms are designed to maximize watch-time rather than learning outcomes [1]. A student searching for a calculus tutorial may receive a video that is far too advanced, in the wrong language, or simply irrelevant to what they need right now. Timmi et al. [1] addressed this *noise problem* using a hybrid content-based and collaborative filtering approach, cutting content rejection rates from 80% to 15%. However, that system treats every student as identical, a beginner and an advanced student searching the same topic receive the same recommendation.

This paper asks a different question: *can a system learn, through trial and error, to select not just what topic to teach but what type of content to deliver and at what difficulty, adapting in real time to each student's current state?* Answering this requires moving from static filtering algorithms to sequential probabilistic decision making.

2. Probabilistic Foundations: Markov Chains

2.1 Stochastic Processes and the Markov Property

A **stochastic process** is a collection of random variables $\{X_t : t \in \mathcal{T}\}$ indexed by time and defined on a common probability space $(\mathcal{F}, \mathbb{P})$. In AdaptED, the student's knowledge state at time t is a random vector $\mathbf{K}_t \in [0,1]^n$, where n is the number of topics being studied. This state evolves as the system delivers content recommendations and the student learns and forgets.

A **Markov chain** is a stochastic process satisfying the *Markov property* [8]: the future is conditionally independent of the past given the present. In other words once you know where you are right now, knowing how you got here provides no additional information about where you are going:

$$\mathbb{P}(\mathbf{K}_{t+1} \mid \mathbf{K}_t, \mathbf{K}_{t-1}, \ldots, \mathbf{K}_0) \;=\; \mathbb{P}(\mathbf{K}_{t+1} \mid \mathbf{K}_t)\,. \tag{1}$$

This is a modeling assumption: the knowledge state $\mathbf{K}_t$ must contain *all* information needed to predict $\mathbf{K}_{t+1}$. This is why the state vector is designed carefully by encoding raw knowledge values and indicators of which topics a student is struggling with and which they have mastered.

2.2 Transition Structure

For a Markov chain with a finite state space $\mathcal{S}$, the dynamics are fully captured by a **transition probability matrix** P, where each entry $P_{ij} = \mathbb{P}(K_{t+1} = s_j \mid K_t = s_i)$ satisfies:

$$P_{ij} \;\geq\; 0 \quad \forall\, i, j \qquad \text{and} \qquad \sum_j P_{ij} \;=\; 1 \quad \forall\, i. \tag{2}$$

Each row sums to 1 because the process must go somewhere at every step.

In AdaptED the transition probabilities are not fixed, it depends on which content the system recommends. A well matched tutorial for a struggling student makes a transition toward higher knowledge highly probable. A challenge problem far above the student's level makes stagnation or decline likely. This action dependence is exactly what elevates the model from a plain Markov chain to a Markov Decision Process, talked about in Section 3.

2.3 The Chapman-Kolmogorov Equations

The n-step transition probabilities satisfy the **Chapman-Kolmogorov equations** [8]:

$$P_{ij}^{(n)} \;=\; \sum_k \; P_{ik}^{(m)} \cdot P_{kj}^{(n-m)} \qquad \text{for any } 0 < m < n. \tag{3}$$

These equations let us reason about long-run behavior by allowing computation of the probability of being in any state after n steps by multiplying shorter transition matrices together. Under a good recommendation system the probability of being in the Mastered state increases over time. Under a bad system, the forgetting curve causes the chain to drift back toward lower knowledge.(Section 6).

3. Markov Decision Processes

3.1 Formal Definition

A **Markov Decision Process** (MDP) extends a Markov chain by introducing an *agent* who selects actions that influence the transition probabilities [8, 11]. An MDP is formally defined as the five-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$ is the **state space**. In AdaptED, $\mathcal{S} \subseteq \mathbb{R}^{15}$, encoding knowledge, struggle flags, and mastery flags across five topics.
- $\mathcal{A}$ is the **action space**, with $|\mathcal{A}| = 150$ discrete actions. Each action is a triple (topic, content type, difficulty).
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the **transition probability function**, where $\mathcal{P}(s' \mid s, a) = \mathbb{P}(K_{t+1} = s' \mid K_t = s, a_t = a)$. The probabilities depend on the action chosen, differeing from a Markov Chain
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the **reward function**. $R_t = \mathcal{R}(s_t, a_t)$ is the scalar feedback the agent receives after each recommendation.
- $\gamma \in [0, 1)$ is the **discount factor**. We use $\gamma = 0.99$, which means future rewards matter almost as much as immediate ones.

3.2 Value Functions and the Bellman Equations

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to probability distributions over actions. The **Q-function** is the expected total discounted reward when action a is taken in state s and policy π is followed thereafter:

$$Q^{\pi}(s, a) \;=\; \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R_t \;\middle|\; s_0 = s,\; a_0 = a\right]. \tag{4}$$

The key insight is that $Q^*(s, a)$ cannot be a fixed number because the student's response to any recommendation is random, its value must be a conditional expectation. This is the

Bellman optimality equation [11]:

$$Q^*(s,a) \;=\; \mathbb{E}\Big[R_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \;\Big|\; s_t = s,\; a_t = a\Big], \tag{5}$$

where the expectation is over the stochastic transition $\mathcal{P}(s' \mid s, a)$. Once Q^* is known, the optimal policy follows directly:

$$\pi^*(s) \;=\; \arg\max_{a \in \mathcal{A}} \; Q^*(s,a). \tag{6}$$

4. The AdaptED System

4.1 State Space and Action Space

The state at time t is a 15-dimensional vector formed by stacking three groups of five:

$$\mathbf{s}_t \;=\; \Big[\, \mathbf{k}_t \;\Big|\; \mathbf{f}_t^{\text{struggle}} \;\Big|\; \mathbf{f}_t^{\text{mastery}} \,\Big] \;\in\; [0,1]^5 \times \{0,1\}^5 \times \{0,1\}^5, \tag{7}$$

where $\mathbf{k}_t \in [0,1]^5$ holds the estimated knowledge for each topic, $\mathbf{f}_t^{\text{struggle}} \in \{0,1\}^5$ flags topics where knowledge is below 0.25 [5], and $\mathbf{f}_t^{\text{mastery}} \in \{0,1\}^5$ flags topics where knowledge exceeds 0.85.

Including the binary flag vectors is a deliberate design choice rooted in probability. Without them, the agent would need to learn on its own that a knowledge value of 0.20 requires a different response than 0.60. By explicitly encoding these categorical facts, the agent can immediately recognize crisis topics and solved topics, which speeds up learning considerably.

Each action $a_t = (\tau, c, d)$ specifies a topic $\tau \in \{1, \ldots, 5\}$, a content type $c \in \{\text{video, example, quiz, etc.}\}$, and a difficulty level $d \in \{0.2,\ 0.4,\ 0.6,\ 0.8,\ 1.0\}$, giving $|\mathcal{A}| = 5 \times 6 \times 5 = 150$ discrete actions.

4.2 The Student Simulator as a Stochastic Process

Because real student interaction data is insufficient to train a reinforcement learning agent from scratch, we build a *student simulator* grounded in educational psychology. The knowledge gain per step follows:

$$\Delta K_t \;=\; \underbrace{\alpha}_{\text{learning rate}} \cdot \underbrace{f_{\text{ZPD}}(d,\, k_\tau)}_{\text{difficulty match}} \cdot \underbrace{E_t}_{\text{engagement}} \cdot \underbrace{\beta_{\text{pref}}}_{\text{preference}} \cdot \underbrace{\mu_c(k_\tau)}_{\text{content type}}, \tag{8}$$

where α is the student's learning rate (shown in Table 1 for each profile), f_{ZPD} is the Zone of Proximal Development factor, E_t is the student's engagement score, β_{pref} is a small boost

when content matches the student's hidden content preference, and μ_c is a content-type multiplier capturing documented pedagogical effectiveness (e.g., a tutorial is 1.2× effective for low-mastery students but only 0.7× for high-mastery students.

The **ZPD factor** operationalizes Vygotsky's Zone of Proximal Development [4, 5], which is the idea that learning is maximized when content is challenging but not overwhelming:

$$f_{\text{ZPD}}(d,\, k) \;=\; \max\!\left(0,\;\; 1 - \frac{|d-k|}{\delta_{\text{ZPD}}}\right), \quad \delta_{\text{ZPD}} = 0.25. \tag{9}$$

When $d = k$ (difficulty perfectly matches current knowledge), $f_{\text{ZPD}} = 1$ and learning is maximized. As the gap grows beyond $\delta_{\text{ZPD}} = 0.25$, the factor drops to zero — content either bores (too easy) or overwhelms (too hard), and no learning occurs.

Table 1: Student profile parameters, grounded in Koedinger et al. [6] (learning rates) and empirical retention studies [7] (forgetting rates).

Profile	**Learning rate α**	**Forgetting rate λ**	**Init. knowledge range**
Fast	0.15	0.005	$[0.20,\ 0.50]$
Average	0.10	0.008	$[0.10,\ 0.40]$
Slow	0.06	0.012	$[0.10,\ 0.35]$
Struggling	0.04	0.015	$[0.05,\ 0.20]$

The **Ebbinghaus forgetting curve** [7] is applied after every step as a stochastic decay:

$$K_{t+1,\,i} \;=\; \underbrace{K_{t,i} + \Delta K_t \cdot \mathbf{1}[i=\tau]}_{\text{learning on topic } \tau} \;-\; \underbrace{\lambda\, \exp(-2\, K_{t,i})}_{\text{forgetting on topic } i}, \quad \forall\, i \in \{1,\ldots,5\}. \tag{10}$$

The exponential term $\exp(-2\,K_{t,i})$ makes forgetting *differential*: topics with low knowledge (small K) decay near their full rate λ, while topics with high knowledge decay at the much slower rate $\lambda\, e^{-2} \approx 0.135\lambda$. This matches Ebbinghaus's empirical finding that weakly encoded memories fade far faster than well consolidated ones [7].

4.3 The Reward Function

The reward at each step is:

$$R_t \;=\; 3.0\,\Delta K_t + 1.5\,E_t + 2.0\,\Delta K_t^{\text{total}} + B_t^{\text{mastery}} - 0.8\,\mathbf{1}[\text{redundant}] - \lambda_r\, \max(0,\; |d - k_\tau| - \delta_{\text{ZPD}})\,, \tag{11}$$

where $\Delta K_t^{\text{total}} = \sum_i (K_{t+1,i} - K_{t,i})$ captures knowledge change across all five topics including forgetting losses, $B_t^{\text{mastery}} = 0.1 \times$ (topics newly mastered) is a milestone bonus, and $\lambda_r = 2.0$.

The dominant term $3.0\,\Delta K_t$ directly rewards knowledge gain. The engagement term $1.5\,E_t$ prevents reward from disengaging recommendations. The ZPD penalty punishes difficulty choices far outside the student's comfort zone, aligning the Bellman-optimal policy with the pedagogically optimal one. A bonus of $+0.5$ is added whenever the agent selects scaffolding content for a topic flagged as struggling.

5. Deep Q-Networks and Probabilistic Policy Learning

5.1 From Tables to Function Approximation

Classical Q-learning stores a table $Q(s, a)$ for every state-action pair and updates it toward the Bellman target. This is impossible when the state space is continuous; our 15-dimensional state has uncountably many possible values. We therefore approximate Q^* with a neural network $Q(s, a;\, \theta)$ [9].

5.2 The Dueling Network Architecture

The **Dueling DQN** architecture. [10], decomposes the Q-function into two separate components:

$$Q(s, a;\, \theta) \;=\; \underbrace{V(s;\, \theta_V)}_{\text{value stream}} + \underbrace{A(s, a;\, \theta_A)}_{\text{advantage stream}} \; - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a';\, \theta_A). \tag{12}$$

The **value stream** $V(s;\, \theta_V)$ estimates a single number: how good is this state overall, no matter what action is taken? The **advantage stream** $A(s, a;\, \theta_A)$ estimates the relative benefit of each specific action compared to the average. The mean subtraction prevents the two streams from learning the same information twice, which keeps the learned representation unique and stable [10].

We split $V(s)$ so that it can be updated from *any* transition that passes through state s, regardless of which action was taken. The value stream can learn from the marginal distribution of states, while the advantage stream requires the joint distribution of state action pairs. Separating them effectively gives the value estimate a larger sample size and speeds up learning.

5.3 Double DQN and Overestimation Bias

Standard Q-learning overestimates action values because $\max_{a'} Q(s', a')$ in the Bellman target introduces upward bias, taking the maximum of noisy estimates always overshoots the true maximum. **Double DQN** [9] fixes this by splitting selection and evaluation across two networks:

$$y_t \;=\; R_t + \gamma\, Q\!\left(s_{t+1},\; \arg\max_{a'} Q(s_{t+1}, a';\, \theta);\; \theta^-\right), \tag{13}$$

where the online network θ selects the action and the target network θ^- (a periodically-frozen copy) evaluates it. Because their errors are partially independent, the bias in the expectation is substantially reduced.

5.4 ε-Greedy Exploration as a Probability Distribution

The agent cannot always pick the action it currently thinks is best. If it never tries new things, it will never discover better strategies. The ε-greedy policy is given by the probability distribution over actions:

$$\pi_\varepsilon(a \mid s) \;=\; \begin{cases} \dfrac{\varepsilon}{|\mathcal{A}|} + (1-\varepsilon) & \text{if } a = \arg\max_{a'} Q(s, a'; \theta), \\ \dfrac{\varepsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{14}$$

With probability ε the agent samples uniformly at random from all 150 actions (exploration); with probability $1 - \varepsilon$ it takes the best-known action (exploitation). We decay ε from 1.0 to 0.05 using the factor 0.9997 per step, so the policy starts near-uniform (high entropy, maximum exploration) and gradually concentrates probability mass on the best-known actions (low entropy, exploitation). This is the exploration-exploitation tradeoff stated formally as a time-varying mixture distribution.

6. Results and Analysis

6.1 Training Convergence

The agent was trained for 1,000 episodes of 60 steps each, with the student profile randomly sampled from {fast, average, slow, struggling} each episode. Figure 1 shows the resulting reward curve. The agent began near-random (reward ≈ 25 at episode 1, consistent with the expected reward under a uniform policy) and converged to a stable plateau of approximately 75–85 by episode 400–500. The peak 50-episode rolling average was 87.03; the final 100-episode average was 80.31.

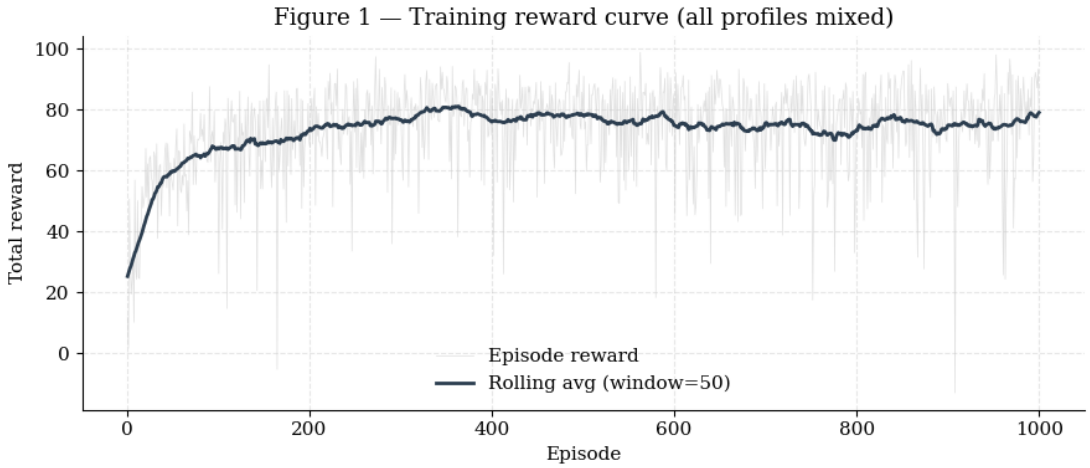

Figure 1: Training reward over 1,000 episodes. The 50-episode rolling average (dark line) rises steadily from ≈ 25 to a plateau near 80, confirming the Dueling DQN is learning an effective recommendation policy. The variance in the raw reward is largely due to random profile sampling.

6.2 Profile-Specific Outcomes

Table 2 summarizes the evaluation results under a fully greedy policy ($\varepsilon = 0$) across 50 episodes per profile. These results show a clear gradient across learner types.

Table 2: Evaluation results by student profile. *Avg Knowledge* is the mean final knowledge across all five topics (scale 0–1). *Mastery Rate* is the fraction of the five topics that reached $k \geq 0.85$ by the end of the episode. *Struggle Fraction* is the fraction of steps spent with knowledge below 0.25.

Profile	**Avg Knowledge**	**Mastery Rate**	**Struggle Frac.**	**Interpretation**
Fast	0.605	0.128	0.125	Strong gains
Average	0.416	0.052	0.334	Consistent
Slow	0.336	0.000	0.438	Improving
Struggling	0.056	0.000	0.848	Forgetting

6.3 The Struggling Learner

The struggling learner outcome with average knowledge of only 0.056 and a struggle fraction of 0.848, can be explained using the stochastic process governing Equation (10). At $k \approx 0.10$, the per-step Ebbinghaus decay on each topic is:

$$\lambda \cdot \exp(-2 \times 0.10) \;=\; 0.015 \cdot e^{-0.2} \;\approx\; 0.0123 \quad \text{per topic, per step.} \tag{15}$$

The maximum possible learning gain for a struggling student under optimal content (a tutorial, perfectly ZPD-matched) is:

$$\Delta K_{\max} \;=\; 0.04 \times 1.0 \times 1.0 \times 1.15 \times 1.20 \times 1.25 \;\approx\; 0.069 \quad \text{on the recommended topic.} \tag{16}$$

However, forgetting applies to *all five topics* every step. The net change in total knowledge per step is therefore at best:

$$\underbrace{0.069}_{\text{gain on topic } \tau} \;-\; \underbrace{5 \times 0.0123}_{\text{forgetting on all topics}} \;=\; 0.069 - 0.0615 \;\approx\; +0.008. \tag{17}$$

This net gain of only +0.008 per step is achieved solely under *perfect* conditions. In practice, the probability of achieving perfect alignment at *every* one of 60 steps is small, so the expected knowledge trajectory declines over the episode. This is a fundamental constraint imposed by the stochastic dynamics of the Ebbinghaus forgetting curve (10).

7. Conclusion

This paper presented AdaptED, a recommendation system built entirely on probabilistic foundations. At its core, modeling the student as a Markov chain allowed the problem to be formalized as an MDP, where every component has a precise probabilistic interpretation: the Bellman equations define the optimal Q-function as a conditional expectation over stochastic transitions, the ε-greedy policy is a time-varying mixture distribution over actions, and experience replay approximates a true expectation through empirical sampling.

The experimental results reveal a clear gradient across learner types. Fast and average learners show consistent knowledge gains, confirming the agent learns to match content type and difficulty to each student's evolving state. The struggling learner outcome as shown in Section 6, even under perfect recommendations the net knowledge gain per step is only $\approx +0.008$, a margin so thin that any suboptimal recommendation drives the expected trajectory below zero. This is a consequence of the forgetting dynamics in Equation (10). Future work will explore longer session, spaced repetition scheduling to reduce forgetting, and validation against real student data from ASSISTments dataset [6].

7. References

[1] M. Timmi, L. Laaouina, A. Jeghal, S. El Garouani, and A. Yahyaouy, "Educational video recommender system," *International Journal of Information and Education Technology*, vol. 14, no. 3, 2024.

[2] Z. Fu, "Integrating reinforcement learning with dynamic knowledge tracing for personalized learning path optimization," *Scientific Reports*, 2025.

[3] C. Piech et al., "Deep knowledge tracing," *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[4] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.

[5] T. Murray and I. Arroyo, "Toward measuring and maintaining the zone of proximal development in adaptive instructional systems," in *Proc. Intelligent Tutoring Systems (ITS)*, 2002.

[6] K. R. Koedinger, J. I. Kim, J. Z. Jia, E. A. McLaughlin, and N. L. Bier, "An astonishing regularity in student learning rate," *Proceedings of the National Academy of Sciences*, 2023.

[7] H. Ebbinghaus, *Memory: A Contribution to Experimental Psychology*. New York: Dover 1964.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[9] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[10] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. ICML*, 2016.

[11] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.

A. Full Simulation Code

Listing 1: AdaptEdu

```
import numpy as np, random
from collections import deque
import torch, torch.nn as nn, torch.optim as optim

NUM_TOPICS = 5
CONTENT_TYPES = ["video","example","quiz","summary","tutorial","challenge"]
DIFFICULTY_LEVELS = [0.2, 0.4, 0.6, 0.8, 1.0]
STATE_SIZE  = NUM_TOPICS * 3          # 15
ACTION_SIZE = NUM_TOPICS * len(CONTENT_TYPES) * len(DIFFICULTY_LEVELS)  # 150
MASTERY_THRESHOLD  = 0.85
STRUGGLE_THRESHOLD = 0.25
ZPD_TOLERANCE      = 0.25

PROFILES = {
    "fast":       {"lr":0.15,"forget":0.005,"init_low":0.20,"init_high":0.50},
    "average":    {"lr":0.10,"forget":0.008,"init_low":0.10,"init_high":0.40},
    "slow":       {"lr":0.06,"forget":0.012,"init_low":0.10,"init_high":0.35},
    "struggling": {"lr":0.04,"forget":0.015,"init_low":0.05,"init_high":0.20},
}
CONTENT_MULTIPLIERS = {
    "video":    lambda k: 0.90,
    "example":  lambda k: 1.00,
    "quiz":     lambda k: 1.10 if k >= 0.40 else 0.65,
    "summary":  lambda k: 0.80,
    "tutorial": lambda k: 1.20 if k <= 0.40 else 0.70,
    "challenge":lambda k: 1.30 if k >= 0.60 else 0.40,
}

class StudentSimulator:
    def __init__(self, profile="average"):
        cfg = PROFILES[profile]
        self.learning_rate   = cfg["lr"]
        self.forgetting_rate = cfg["forget"]
        self.knowledge = np.random.uniform(cfg["init_low"],cfg["init_high"],
                                           size=NUM_TOPICS)
        self.preference = random.choice(CONTENT_TYPES)

    def step(self, topic, content_type, difficulty):
        k = self.knowledge[topic]
        zpd_factor = max(0.0, 1.0 - abs(difficulty - k) / ZPD_TOLERANCE)
        c_mult     = CONTENT_MULTIPLIERS[content_type](k)
        pref_boost = 1.15 if content_type == self.preference else 1.0
        engagement = 1.0 if abs(difficulty - k) <= ZPD_TOLERANCE else 0.30
        delta_k    = self.learning_rate * zpd_factor * engagement \
                     * pref_boost * c_mult
        if k < STRUGGLE_THRESHOLD and content_type in \
                ["tutorial","example","summary"]:
            delta_k *= 1.25
        elif k < STRUGGLE_THRESHOLD and content_type in ["challenge","quiz"]:
            delta_k *= 0.55
        prev_knowledge = self.knowledge.copy()
```

```
        self.knowledge[topic] = float(np.clip(k + delta_k, 0.0, 1.0))
        # Ebbinghaus forgetting
        decay = self.forgetting_rate * np.exp(-self.knowledge * 2.0)
        self.knowledge = np.clip(self.knowledge - decay, 0.0, 1.0)
        return self.knowledge.copy(), delta_k, engagement, \
               abs(difficulty - k), prev_knowledge

class LearningEnv:
    def __init__(self, profile="average"):
        self.profile = profile
        self.student  = StudentSimulator(profile)
        self.prev_action_idx = None
        self.prev_knowledge  = self.student.knowledge.copy()

    def reset(self):
        self.student = StudentSimulator(self.profile)
        self.prev_action_idx = None
        self.prev_knowledge  = self.student.knowledge.copy()
        return self._state()

    def _state(self):
        k = self.student.knowledge
        return np.concatenate([k,
            (k < STRUGGLE_THRESHOLD).astype(float),
            (k >= MASTERY_THRESHOLD).astype(float)])

    def decode_action(self, idx):
        NC, ND = len(CONTENT_TYPES), len(DIFFICULTY_LEVELS)
        return (idx // (NC*ND),
                CONTENT_TYPES[(idx % (NC*ND)) // ND],
                DIFFICULTY_LEVELS[(idx % (NC*ND)) % ND])

    def step(self, action_idx):
        topic, ctype, diff = self.decode_action(action_idx)
        next_k, dk, eng, dg, prev_k = self.student.step(topic, ctype, diff)
        dkt  = float(np.sum(next_k - prev_k))
        mb   = 0.1 * max(0, int(np.sum(next_k  >= MASTERY_THRESHOLD))
                          - int(np.sum(prev_k  >= MASTERY_THRESHOLD)))
        red  = 1.0 if self.prev_action_idx == action_idx else 0.0
        zpd  = 2.0 * max(0.0, dg - ZPD_TOLERANCE)
        reward = 3.0*dk + 1.5*eng + 2.0*dkt + mb - 0.8*red - zpd
        if self.student.knowledge[topic] < STRUGGLE_THRESHOLD and \
                ctype in ["tutorial","example","summary"]:
            reward += 0.5
        self.prev_action_idx = action_idx
        self.prev_knowledge  = next_k.copy()
        return self._state(), reward, bool(np.all(next_k >= MASTERY_THRESHOLD))

class DuelingDQN(nn.Module):
    def __init__(self):
        super().__init__()
        self.shared = nn.Sequential(
            nn.Linear(STATE_SIZE, 128), nn.ReLU(),
            nn.Linear(128, 128),        nn.ReLU())
        self.value_head     = nn.Linear(128, 1)
```

```
        self.advantage_head = nn.Linear(128, ACTION_SIZE)

    def forward(self, x):
        h = self.shared(x); v = self.value_head(h); a = self.advantage_head(h)
        return v + (a - a.mean(dim=1, keepdim=True))

class Agent:
    def __init__(self):
        self.online = DuelingDQN(); self.target = DuelingDQN()
        self.target.load_state_dict(self.online.state_dict())
        self.optimizer = optim.Adam(self.online.parameters(), lr=5e-4)
        self.memory    = deque(maxlen=10000)
        self.gamma = 0.99; self.epsilon = 1.0
        self.epsilon_decay = 0.9997; self.epsilon_min = 0.05
        self.batch_size = 64

    def act(self, state):
        if random.random() < self.epsilon:
            return random.randint(0, ACTION_SIZE - 1)
        with torch.no_grad():
            q = self.online(torch.FloatTensor(state).unsqueeze(0))
        return int(q.argmax().item())

    def store(self, s, a, r, s2, done):
        self.memory.append((s, a, r, s2, done))

    def train_step(self):
        if len(self.memory) < self.batch_size: return
        batch = random.sample(self.memory, self.batch_size)
        s, a, r, s2, done = zip(*batch)
        s    = torch.FloatTensor(np.array(s))
        s2   = torch.FloatTensor(np.array(s2))
        a    = torch.LongTensor(a); r = torch.FloatTensor(r)
        done = torch.FloatTensor(done)
        cq   = self.online(s).gather(1, a.unsqueeze(1)).squeeze()
        with torch.no_grad():
            ba = self.online(s2).argmax(dim=1)   # Double DQN
            tq = r + self.gamma*(1-done)* \
                 self.target(s2).gather(1, ba.unsqueeze(1)).squeeze()
        loss = nn.HuberLoss()(cq, tq)
        self.optimizer.zero_grad(); loss.backward(); self.optimizer.step()
        self.epsilon = max(self.epsilon_min, self.epsilon*self.epsilon_decay)

    def sync_target(self):
        self.target.load_state_dict(self.online.state_dict())

def train(episodes=1000, steps=60, sync_every=10):
    agent = Agent(); profiles = list(PROFILES.keys()); ep_rewards = []
    for ep in range(episodes):
        profile = random.choice(profiles)
        env = LearningEnv(profile=profile); state = env.reset(); total_r = 0.0
        for _ in range(steps):
            action = agent.act(state)
            next_state, reward, done = env.step(action)
            agent.store(state, action, reward, next_state, float(done))
```

```
            agent.train_step(); state = next_state; total_r += reward
            if done: break
        if ep % sync_every == 0: agent.sync_target()
        ep_rewards.append(total_r)
        if (ep+1) % 100 == 0:
            print(f"Ep {ep+1:>4} | Profile: {profile:<10} | "
                  f"Reward: {total_r:>7.2f} | "
                  f"Avg(last 100): {np.mean(ep_rewards[-100:]):>6.2f} | "
                  f"Avg Know: {np.mean(env.student.knowledge):.3f} | "
                  f"eps: {agent.epsilon:.3f}")
    return agent, ep_rewards

if __name__ == "__main__":
    agent, rewards = train()
```

B. Numerical Results

B.1 Final Evaluation Summary

```
======================================================================
Profile       Avg Knowledge  Mastery Rate  Struggle Frac
----------------------------------------------------------------------
Fast                  0.605         0.128          0.125
Average               0.416         0.052          0.334
Slow                  0.336         0.000          0.438
Struggling            0.056         0.000          0.848
======================================================================

Overall training reward (last 100 eps): 80.31
Peak rolling avg reward               : 87.03
```